



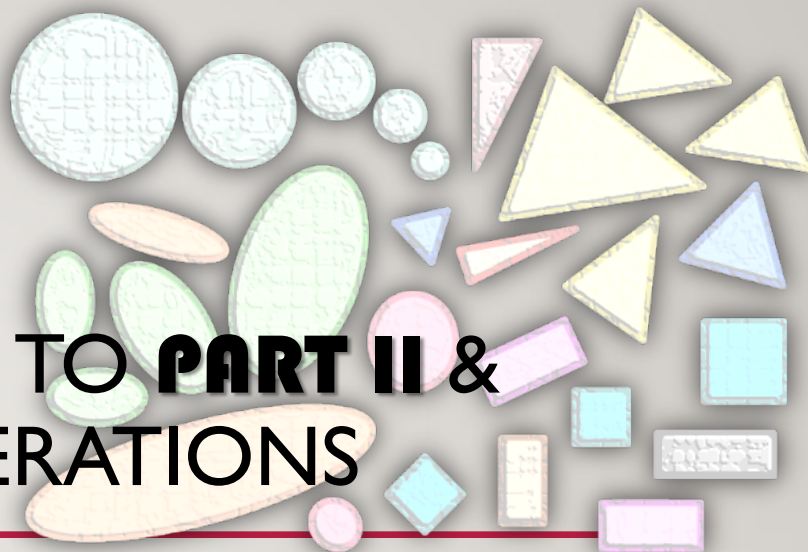
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tamaki Makaurau
NEW ZEALAND

LECTURE 18

INTRODUCTION TO **PART II** & DESIGN CONSIDERATIONS

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH

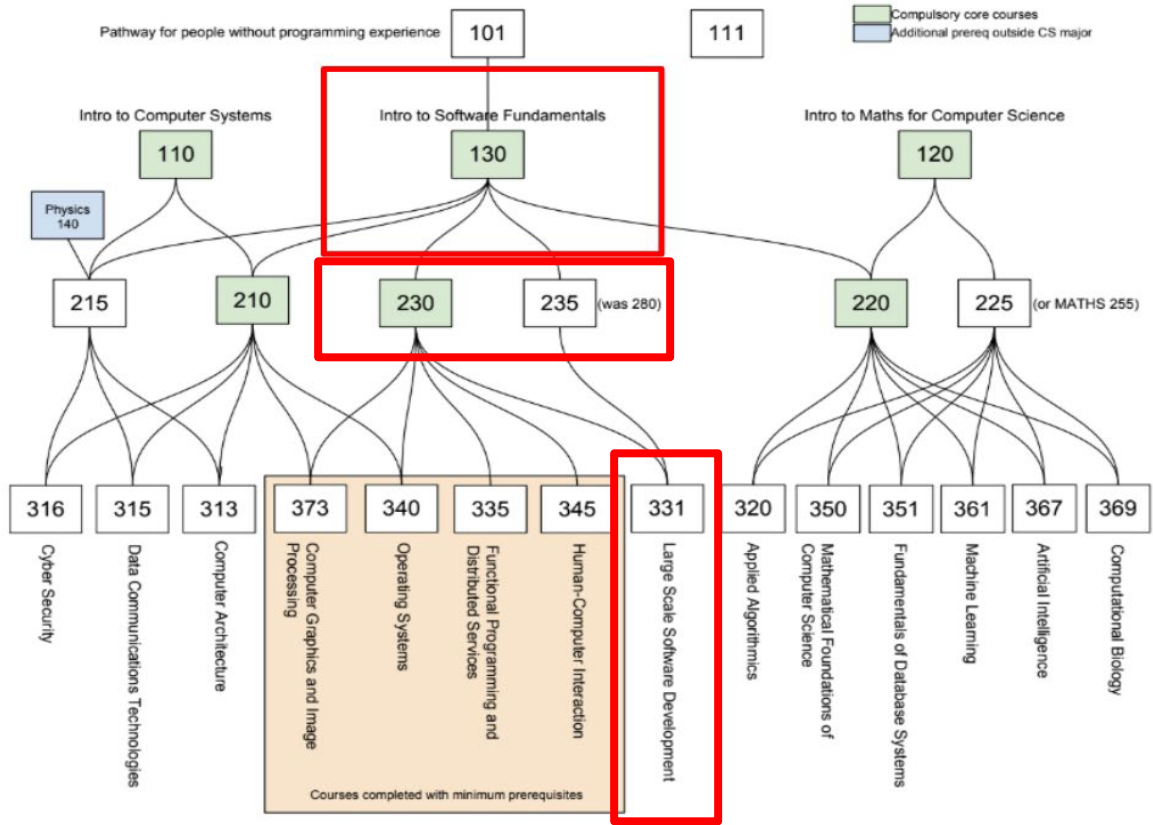


AGENDA

- Build-up
- Case Studies
 - Part I knowledge – Revising fundamental Java/OOP knowledge
 - Visualizing domain classes interactions
 - Design Considerations



Computer Science Major



BUILD UP – LEARNING OUTCOMES

On successfully completing this course, you will be able to:

- **OO Programming:** describe and use the features typically offered by an object-oriented programming language, including support for classes, visibility, inheritance, interfaces, polymorphism and dynamic binding
- **OO Design:** explain and apply key design principles of object-oriented software development, including separation of concerns, abstraction, information hiding, programming to interfaces, coupling and cohesion, resilience to change, and reuse
- create simple OO design models
- **Design Patterns:** develop a small graphical user interface (GUI) based application in an object-oriented programming language; Assess and improve software design decisions based on object-oriented design objectives and principles
- **Frameworks:** describe important concepts of programming frameworks, including APIs, inversion of control, concurrency and event-driven programming

BUILD UP – LOOKING AHEAD

- **Topics (*high-level*)**
 - Modelling with the UML
 - Object-oriented design objectives and principles
 - Software design patterns
 - Object-oriented Java GUI frameworks
 - Concurrency (with multithreading)
- **COMPSCI 230**
 - **15-point course**
 - Study **10 hours per week**
- **Assignments (A3)**
- **Lectures**
 - **3 hours each week**
 - **Study** the lecture **reading material**, **watch recordings**, **solve exercises** – see **Canvas Modules**
 - Engage in **in-class activities**
 - **Write notes** in class/labs/own time
 - Maintain *learning journals* (see my **JCGs** on **Canvas Modules**)
 - Talk to me if you need help
- **Labs**
 - 6 Assessed/*marked* labs in Part 2
 - Assignment support
- **No Test**

BUILD UP – ASSESSMENTS (REMAINING)

- **Assignments** (6%)

- A3

- **Examination** (50%)

- Part – 1 (15%)
- Part – 2 (35%)



- **A2** (due week 9)

- **A3** (due week 12)

Focus – Design Patterns and MVC architecture

COURSE SCHEDULE – PART 2

Week no. <i>starting</i>	Monday	Tuesday	Thursday	Lab
7 <i>24 Apr</i>	Introduction to Part-2	Software Modelling with UML – 1	Software Modelling with UML – 2	Lab07: Exceptions, FileIO & Simple GUI
8 <i>1 May</i>	Software Design Objectives and SOLID Principles	Design Patterns: Introduction and Composite Pattern	Template Method Pattern	Lab08: Class diagrams & GUI
9 <i>8 May</i>	Observer Pattern	Java GUI - I AWT Framework and Event handling	Java GUI - II Swing Framework	Lab09: Composite & Template Method
10 <i>15 May</i>	Adaptor Pattern (TreeModel ListModel)	Model-View-Controller Pattern (DocumentModel, TableModel)	Live coding session-1 (In-class exercises)	Lab10: Observer and ListModel
11 <i>22 May</i>	Singleton, Immutable and Factory Patterns	Java Multithreading	Multithreading and GUI	Lab11: TreeModel
12 <i>29 May</i>	Live coding session-2 (In-class exercises)	Part-2 Review	Course Conclusion	Lab10: TableModel, Singleton, Factory

ONCE UPON A TIME, THERE WAS PART I ! (10 MINS)

- Open the **Google Form link (shared on Piazza and also mentioned below)** and fill in the form about your understanding of Part-I topics.
- <https://forms.gle/qgeit22nuFkEv45H6>
 - Class
 - Object
 - Inheritance
 - Polymorphism
 - Dynamic binding
 - Abstract class
 - Interface
 - Nested classes
 - Exception handling
 - Information hiding (encapsulation)
 - Generics

COMPSCI230 Once upon a time in Part 1!

This form is designed to record students' inputs regarding the first-half topics of COMPSCI 230 S1 2023.

Expected time required: 20-30 mins

Note: All responses are completely "anonymised"

Reflect on your understanding of the following Part-1 topics of COMPSCI 230. You may write as much as you like for each topic. This may also be the appropriate start for your organized note-taking and journal writing, if you decide to do it.

For example,

John is one of the students in COMPSCI 230 who worked hard on Part-1 topics but had the following reflections on their learning of the concept of "class":

"I was quite confused about the notion of "class", especially about how to identify and design one given a problem domain. To be honest, I am still not sure about how a class differs from an object. Are they one and the same thing? the class notes say - a class is a collection of objects, but I also read online - a class is a set of properties that represent an object. Which one should I follow? I am afraid that as this is a software design and development course, if they give me a problem statement and ask to identify required classes and objects, I am going to struggle."

Class

Long-answer text

Object

Long-answer text

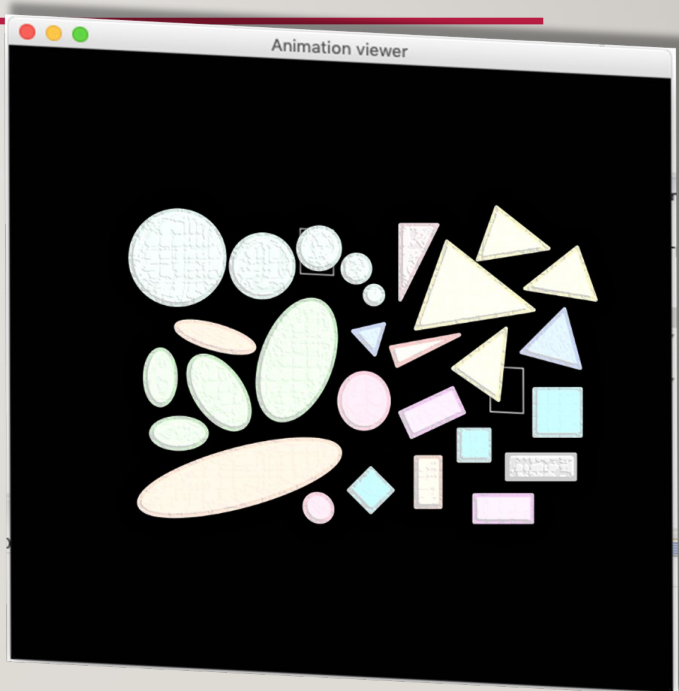
ONCE UPON A TIME, THERE WAS PART I ! (10 MINS)

- You have been given the responsibility for designing a **transport management game**. For each key **OOD feature** listed below and the entities shown on the right, **define the feature** and **describe at least one instance** where each of these features can be used to implement the physical entities shown on the right.
 - Class
 - Object
 - Inheritance
 - Polymorphism
 - Dynamic binding
 - Abstract class
 - Interface
 - Nested classes
 - Exception handling
 - Information hiding (encapsulation)
 - Generics



A DESIGN PROBLEM

- Consider an animation where **shapes** move around a 2-dimensional dark space; bounce off the enclosing boundaries
 - **Shapes** are characterised as follows:
 - Each **Shape** has a velocity, direction, location, and size (bounding box)
 - **Shapes** move and bounce according to their velocity and direction, and within the confines of the 2 dimensional space
 - **Shapes** are painted according to their type (e.g. rectangle, circle etc.), location and size
 - Initially, two kinds of **Shapes** are required – rectangles and ovals – but the set of shape types should be extensible
- Assume the existence of two classes:
 - **Graphics**, provides primitives for drawing on the screen
 - **Timer**, generates tick events at a specified frequency



DEVELOPING THE DESIGN (10 MINS)

- Construct object-oriented designs for the problem. The designs should show:
 - **Key classes** and how they're related
 - **Key objects** and how they interact
- Express the solutions using **informal diagrams, annotations** and **source code fragments**

OBJECT-ORIENTED DESIGN CONSIDERATIONS

• Design vs Code

- **Design** represents one of the many solutions for a software development problem
- A design comprises one or more minor or major decisions about a given software feature (e.g. method, class, variable, information hiding, inheritance, etc.)
- **Code** is the **implementation** of design
- Example
 - factorial program designs

```
import java.util.Scanner;
class Factor1
{
    public static void main(String args[])
    {
        long n,fact=1;

        n=Long.parseLong(args[0]);

        for(int i=1;i<=n;i++)
        {
            fact=fact*i;
        }

        System.out.println("fact="+fact);
    }
}
```

```
class Factor1
{
    public static void main(String args[])
    {
        long n,fact=0;

        n=Long.parseLong(args[0]);

        fact=factCal(n);

        System.out.println("fact="+fact);
    }

    static long factCal(long x)
    {
        long fact=1;
        for(int i=1;i<=x;i++)
        {

            fact=fact*i;

        }
        return fact;
    }
}
```

```
class Factor1
{
    public static void main(String arg[])
    {
        long n;

        Scanner sc=new Scanner(System.in);

        System.out.println("enter number");

        n=sc.nextLong();

        long f=Factor1.fact(n);

        System.out.println("factorial="+f);
    }

    static long fact(long n)
    {
        if(n<=0)
            return 1;

        return Factor1.fact(n-1)*n;
    }
}
```

ARTEFACTS FOR DESIGN #I

```
public class Shape {  
    private int _x;  
    private int _y;  
    private int _deltaX;  
    private int _deltaY;  
    private int _width;  
    private int _height;  
    private int _type;
```

```
    private static final int DEFAULT_X_POS = 0;  
    private static final int DEFAULT_Y_POS = 0;  
    private static final int DEFAULT_DELTA_X = 5;  
    private static final int DEFAULT_DELTA_Y = 5;  
    private static final int DEFAULT_HEIGHT = 35;  
    private static final int DEFAULT_WIDTH = 25;
```

```
    public static final int RECTANGLE = 0;  
    public static final int OVAL = 1;
```

```
    public Shape( int type, int x, int y ) {  
        _type = type;  
        _x = x;  
        _y = y;  
        _deltaX = DEFAULT_DELTA_X;  
        _deltaY = DEFAULT_DELTA_Y;  
        _width = DEFAULT_WIDTH;  
        _height = DEFAULT_HEIGHT;  
    }  
    ...  
} // End of class definition
```

```
public class Shape {  
    /** Moves a Shape instance within a two-dimensional space  
     * @param width the enclosing world's width  
     * @param height the enclosing world's height  
     */  
    public void move( int width, int height ) {  
        // Implementation omitted.  
    }  
  
    /** Displays a shape on the screen  
     * @param g the Graphics object supplied to a Shape as  
     *         part of a paint() request  
     */  
    public void paint( Graphics g ) {  
        if( _type == RECTANGLE )  
            g.drawRect( _x, _y, _width, _height);  
        else  
            g.drawOval( _x, _y, _width, _height );  
    }  
}
```

SOME DESIGN CONSIDERATIONS...

- Which other domain **classes and objects** do we require?
- The Shape class definition **encapsulates** state. What are the benefits of **information hiding** in the context of class Shape?
- Can you think of some **inheritance** relationships that will help solve the problem?
- Where will **dynamic binding** come handy?
- Can you relate this design problem to ***what you learnt about those OO features in Part-I?***

INHERITANCE - “GOOD” CLASS HIERARCHIES

- Class hierarchies involve **general superclasses towards the top** with specialised subclasses towards the bottom
- Superclasses offer concept and/or code **reuse**
- Very general concepts are represented as **abstract classes** that can't be instantiated
- **Common state and behaviour are pushed up** the hierarchy and defined/implemented by superclasses
- Subclasses can override inherited method implementations and typically also implement inherited method definitions (i.e. **abstract methods**)
- The **is-a** relationship should generally hold between a subclass and its superclass

ARTEFACTS FOR DESIGN #I

```
public class Shape {  
    ...  
    public static final int RECTANGLE = 101;  
    public static final int OVAL = 102;  
    public static final int TRIANGLE = 103;  
    public static final int HEXAGON = 104;  
    public static final int HEXAGON_WITH_MESSAGE = 105;  
    public static final int DYNAMIC_RECTANGLE = 106;  
    public static final int OVAL_WITH_MESSAGE = 107;  
    public static final int CIRCLE = 108;  
    public static final int NOISY_CIRCLE = 109;  
    public static final int SOLID_RECTANGLE = 110;  
}
```

```
public void paint( Graphics g ) {  
    switch( _type ) {  
        case RECTANGLE:  
            g.drawRect( _x, _y, _width, _height );  
            break;  
        case OVAL:  
            g.drawOval( _x, _y, _width, _height );  
            break;  
        ...  
    }  
    ...  
}
```

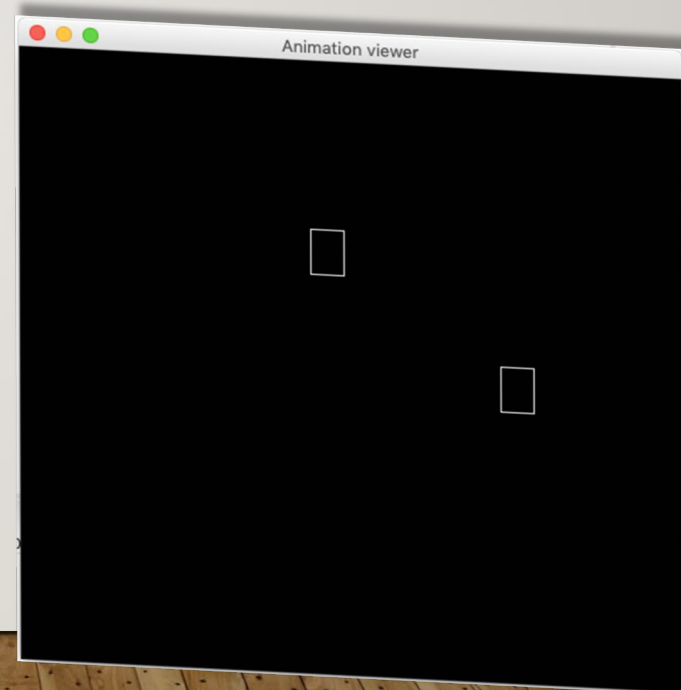
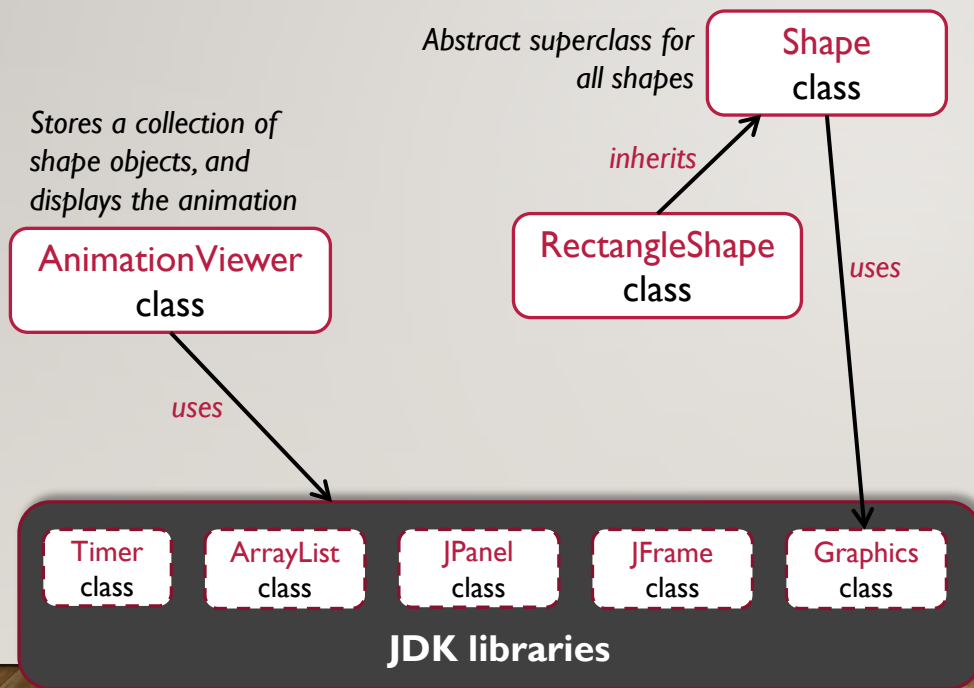
ARTEFACTS FOR DESIGN #2

```
public abstract class Shape {  
    protected int _x;  
    protected int _y;  
    protected int _deltaX;  
    protected int _deltaY;  
    protected int _width;  
    protected int _height;  
    ...  
}
```

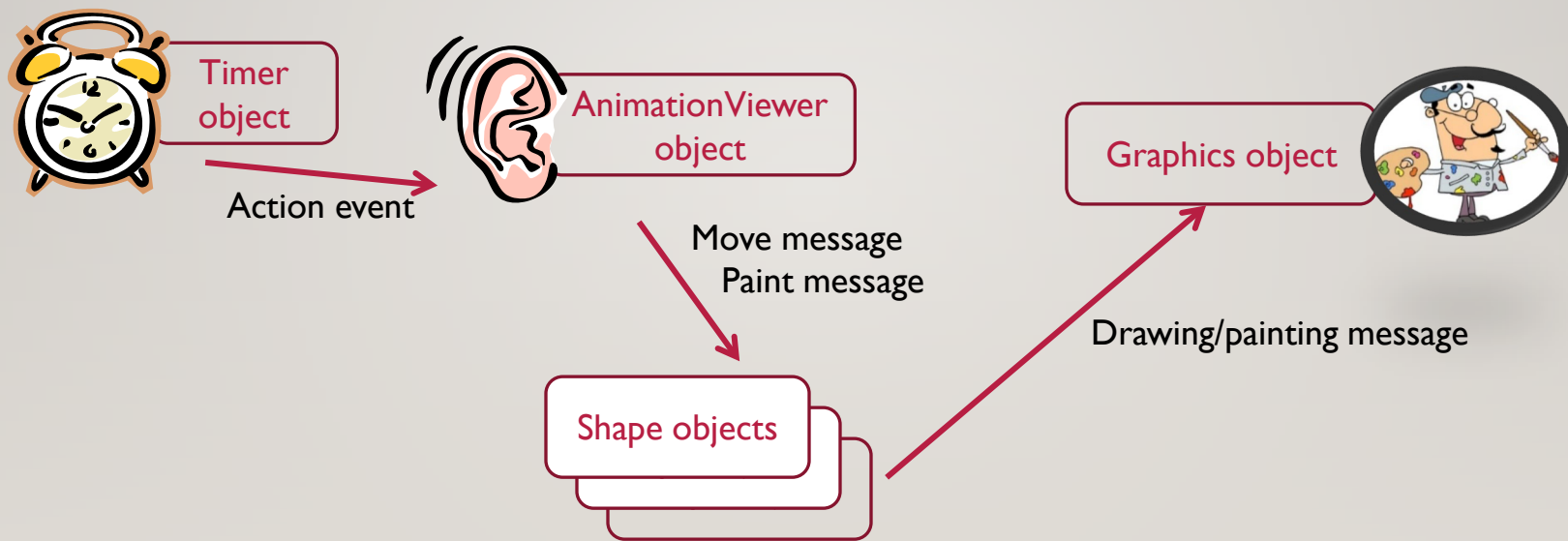
```
private static final int DEFAULT_X_POS = 0;  
private static final int DEFAULT_Y_POS = 0;  
private static final int DEFAULT_DELTA_X = 5;  
private static final int DEFAULT_DELTA_Y = 5;  
private static final int DEFAULT_HEIGHT = 35;  
private static final int DEFAULT_WIDTH = 25;  
...  
}
```

```
/**  
 * Moves a Shape instance within a two-dimensional space  
 * @param width the enclosing world's width  
 * @param height the enclosing world's height  
 */  
public void move( int width, int height ) {  
    // Implementation omitted.  
}  
  
/**  
 * Displays a shape on the screen  
 * @param g the Graphics object supplied to a Shape as  
 *           part of a paint() request  
 */  
public abstract void paint( Graphics g );  
}
```

THE CLASS DESIGN MODEL



OBJECT INTERACTION MODEL



SUGGESTED TASKS

- Work through the design problems / **class exercises** discussed in this lecture

GENERAL SUGGESTIONS

- Part-2 presents a more **systematic and professional way of writing computer programs** (in fact *software*)
- **Online Zoom labs/sessions are ideal for asking** questions and having discussions
- **Assignments** are *linearly dependent* ($A1 \Rightarrow A2 \Rightarrow A3$)
- There is **no single prescribed text book for Part-2**, but it does not mean there is no content around (see *Readings for each lecture*)
- Getting to the *right* content – **Google carefully!**
-and...

ASK QUESTIONS !

- **Never hesitate to approach** our teaching team (and your class mates) for:
 - asking questions
 - clarifying facts
 - discussions

REFERENCES

➤ [Books]

- Chapter-2 Overview of Object Orientation, Timothy Lethbridge and Robert Leganaire, Object-Oriented Software Engineering: Practical Software Development using UML and Java, McGraw-Hill publications.
- *Java How to Program*, Ch 16, 20, Dietel and Dietel Publications.

➤ [Online]

- <https://docs.oracle.com/javase/17/docs/>

UP NEXT...

- Object-Oriented Software Modelling